

Exp no:7**Endorsement Policy Configuration in Hyperledger Fabric****Aim**

To configure and implement endorsement policies in Hyperledger Fabric and verify how multiple organizations endorse transactions before they are committed to the blockchain ledger.

1. Go to the test network

None

```
cd ~/fabric-dev/fabric-samples/test-network
```

2. Stop the previous experiment

None

```
./network.sh down
```

Then start a fresh network:

None

```
./network.sh up createChannel -ca
```

```
g2MSPanchors.tx
026-09-08 18:12:44.224 IST 0001 INFO [channelCmd] InitCmdFactory → Endorser and orderer connections initialized
026-09-08 18:12:44.231 IST 0002 INFO [channelCmd] update → Successfully submitted channel update
anchor peer set for org 'Org2MSP' on channel 'mychannel'
channel 'mychannel' joined
```

3. Set the Fabric environment

None

```
export PATH=${PWD}/../bin:$PATH
export FABRIC_CFG_PATH=$PWD/../config/

export CORE_PEER_TLS_ENABLED=true
```

```

export CORE_PEER_LOCALMSPID="Org1MSP"

export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt

export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp

export CORE_PEER_ADDRESS=localhost:7051

export
ORDERER_CA=${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem

```

Part A — OR Endorsement Policy

None

```
OR('Org1MSP.peer', 'Org2MSP.peer')
```

meaning **either organization can endorse the transaction**.

4. Deploy chaincode with OR policy

None

```

./network.sh deployCC \
-ccl javascript \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript \
-ccep "OR('Org1MSP.peer', 'Org2MSP.peer')"

```

```
committed chaincode definition for chaincode 'basic' on channel 'mychannel'
Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscv, Approvals: [Org1MSP: true, Org2MSP: true]
Query chaincode definition successful on peer0.org2 on channel 'mychannel'
Chaincode initialization is not required
```

5. Verify the committed chaincode

None

```
peer lifecycle chaincode querycommitted \
-C mychannel
```

```
—(ram@ram)-[~/fabric-dev/fabric-samples/test-network]
$ peer lifecycle chaincode querycommitted \
-C mychannel
Committed chaincode definitions on channel 'mychannel':
Name: basic, Version: 1.0, Sequence: 1, Endorsement Plugin: escc, Validation Plugin: vscv
```

6. Initialize the ledger

Use both peers for the transaction:

None

```
peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "$ORDERER_CA" \
-C mychannel \
-n basic \
-c '{"function":"InitLedger","Args":[]}' \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles
"${PWD}/organizations/peerOrganizations/org2.example.com/tlsca
/tlsca.org2.example.com-cert.pem" \
--waitForEvent
```

```
ram@ram: ~/fabric-dev/fabric-samples/test-network
$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "$ORDERER_CA" \
-c mychannel \
-n basic \
-c '{"function":"InitLedger","Args":[]}' \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "${PWD}/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.example.com-cert.pem" \
--waitForEvent
2026-09-08 18:15:03.249 IST 0001 INFO [chaincodeCmd] ClientWait → txid [851a4171c4413dd4c6afac9ce9000f044fea3de50e8874fece92c724a096dc9b] committed with status (VALID) at localhost:7051
2026-09-08 18:15:03.250 IST 0002 INFO [chaincodeCmd] ClientWait → txid [851a4171c4413dd4c6afac9ce9000f044fea3de50e8874fece92c724a096dc9b] committed with status (VALID) at localhost:9051
2026-09-08 18:15:03.251 IST 0003 INFO [chaincodeCmd] chaincodeInvokeOrQuery → Chaincode invoke successful. result: status:200
```

7. Query the ledger

```
None
peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["GetAllAssets"]}'
```

```
ram@ram: ~/fabric-dev/fabric-samples/test-network
$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args":["GetAllAssets"]}'
[{"AppraisedValue":300,"Color":"blue","ID":"asset1","Owner":"Tomoko","Size":15,"docType":"asset"}, {"AppraisedValue":400,"Color":"red","ID":"asset2","Owner":"Brad","Size":15,"docType":"asset"}, {"AppraisedValue":500,"Color":"green","ID":"asset3","Owner":"Jin Soo","Size":10,"docType":"asset"}, {"AppraisedValue":600,"Color":"yellow","ID":"asset4","Owner":"Max","Size":10,"docType":"asset"}, {"AppraisedValue":700,"Color":"black","ID":"asset5","Owner":"Adriana","Size":15,"docType":"asset"}, {"AppraisedValue":800,"Color":"white","ID":"asset6","Owner":"Michel","Size":15,"docType":"asset"}]
```

Part B — AND Endorsement Policy

Now we need to demonstrate the second policy.

8. Stop the network

```
None
./network.sh down
```

9. Start a fresh network

```
None
./network.sh up createChannel -ca
```

```
Org2MSPanchors.LX
2026-09-08 18:17:14.055 IST 0001 INFO [channelCmd] InitCmdFactory → Endorser and orderer connections initialized
2026-09-08 18:17:14.063 IST 0002 INFO [channelCmd] update → Successfully submitted channel update
Anchor peer set for org 'Org2MSP' on channel 'mychannel'
channel 'mychannel' joined
export FABRIC_CFG_PATH=$PWD/../config/
```

10. Set the environment again

None

```
export PATH=${PWD}/../bin:$PATH
export FABRIC_CFG_PATH=${PWD}/../config/

export CORE_PEER_TLS_ENABLED=true
export CORE_PEER_LOCALMSPID="Org1MSP"

export
CORE_PEER_TLS_ROOTCERT_FILE=${PWD}/organizations/peerOrganizations/org1.example.com/peers/peer0.org1.example.com/tls/ca.crt

export
CORE_PEER_MSPCONFIGPATH=${PWD}/organizations/peerOrganizations/org1.example.com/users/Admin@org1.example.com/msp

export CORE_PEER_ADDRESS=localhost:7051

export
ORDERER_CA=${PWD}/organizations/ordererOrganizations/example.com/orderers/orderer.example.com/msp/tlscacerts/tlsca.example.com-cert.pem
```

11. Deploy chaincode with AND policy

This is the important command:

None

```
./network.sh deployCC \
-ccl javascript \
-ccn basic \
-ccp ../asset-transfer-basic/chaincode-javascript \
-ccep "AND('Org1MSP.peer','Org2MSP.peer')"
```

This means:

None

Org1 + Org2

↓

Both must endorse

↓

Transaction can become valid

12. Create asset25

None

```
peer chaincode invoke \  
-o localhost:7050 \  
--ordererTLSHostnameOverride orderer.example.com \  
--tls \  
--cafile "$ORDERER_CA" \  
-C mychannel \  
-n basic \  
-c \  
'{"function": "CreateAsset", "Args": ["asset25", "Blue", "10", "Arun", "1200"]}' \  
--peerAddresses localhost:7051 \  
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \  
--peerAddresses localhost:9051 \  
--tlsRootCertFiles \  
"${PWD}/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.example.com-cert.pem" \  
--waitForEvent
```

Because the policy is:

None

```
AND('Org1MSP.peer', 'Org2MSP.peer')
```

both peers are supplied as endorsers.

```
(ram@ram): ~/fabric-dev/fabric-samples/test-network
$ peer chaincode invoke \
-o localhost:7050 \
--ordererTLSHostnameOverride orderer.example.com \
--tls \
--cafile "ORDERER_CA" \
-C mychannel \
-n basic \
-c '{"function": "CreateAsset", "Args": ["asset25", "Blue", "10", "Arun", "1200"]}' \
--peerAddresses localhost:7051 \
--tlsRootCertFiles "$CORE_PEER_TLS_ROOTCERT_FILE" \
--peerAddresses localhost:9051 \
--tlsRootCertFiles "${PWD}/organizations/peerOrganizations/org2.example.com/tlsca/tlsca.org2.example.com-cert.pem" \
--waitForEvent
2020-09-08 18:18:53.531 IST 0001 INFO [chaincodeCmd] ClientWait -> txid [80967ffc6f265c000927f7b3407f84dda8dd16196d5d069ebfdcc5ab81194e5] committed with status (VALID) at localhost:7051
2020-09-08 18:18:53.531 IST 0002 INFO [chaincodeCmd] ClientWait -> txid [80967ffc6f265c000927f7b3407f84dda8dd16196d5d069ebfdcc5ab81194e5] committed with status (VALID) at localhost:9051
2020-09-08 18:18:53.532 IST 0003 INFO [chaincodeCmd] chaincodeInvokeQuery -> Chaincode invoke successful. result: status:200 payload: '{"ID":"asset25","Color":"Blue","Size":10,"Owner":"Arun","AppraisedValue":1200}'
```

13. Verify asset25

None

```
peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args": ["ReadAsset", "asset25"]}'
```

```
(ram@ram): ~/fabric-dev/fabric-samples/test-network
$ peer chaincode query \
-C mychannel \
-n basic \
-c '{"Args": ["ReadAsset", "asset25"]}'
{"AppraisedValue":1200,"Color":"Blue","ID":"asset25","Owner":"Arun","Size":10}
```